



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algoritmusok és Adatszerkezetek I.

Pusztai Gábor

Algoritmusok és Alkalmazásaik Tanszék
Informatikai Kar
ELTE - Eötvös Loránd Tudományegyetem, Budapest

2026. február 23.

1 Láncolt lista - Egyirányú

- Egyszerű, egyirányú láncolt lista (S1L)
- Iterálás és Keresés
- Beszúrás
- Törlés

2 Fejelemes lista - Egyirányú

- Fejelemes egyirányú láncolt lista (H1L)
- Beszúrás

3 Feladatok

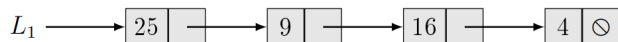
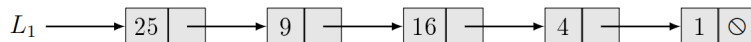
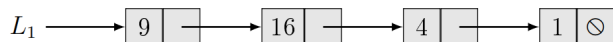
- S1L lista megfordítása
- H1L szétfűzése két listába
- Primszita egyszerű listán

Láncolt lista - Egyirányú

- A **láncolt lista** olyan adatszerkezet, amelyben az objektumok lineáris sorrendben követik egymást. A tömböknél a lineáris sorrendet a tömbindexek határozzák meg, ezzel szemben a láncolt listákban mutatók valósítják meg az elemek lineáris elrendezettségét: a lista minden objektuma tartalmaz egy mutatót, amely a következő objektumra mutat. *Forrás : Új algoritmusok by Cormen et. al.*
- Típusai:
 - Egyszerű, egyirányú láncolt lista (S1L)
 - Fejelemes egyirányú láncolt lista (H1L)
 - Ciklikus egyirányú lista (C1L)
 - Egyszerű, kétirányú lista (S2L)
 - Ciklikus kétirányú lista (C2L) fejelemmel

Egyszerű, egyirányú láncolt lista (S1L)

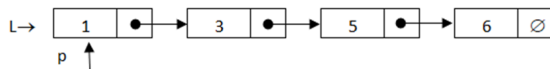
$L_1 = \emptyset$



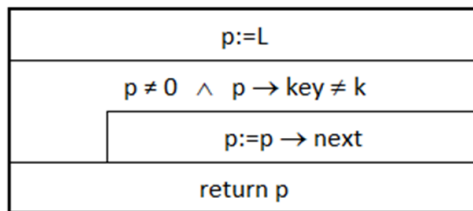
- Minden listaelem rendelkezik a saját adatával és egy mutatóval, amely a lista következő elemére mutat. Ha ez az utolsó elem, akkor a mutató *NULL*.
- **L** az első elemre mutat, de **L** nem maga egy elem, és nem is egy E1 típus.

E1
$+key : \mathcal{T}$
$\dots$ // satellite data may come here
$+next : E1^*$
$+E1() \{ next := \emptyset \}$

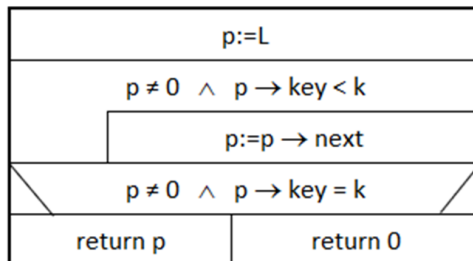
Egyszerű, egyirányú láncolt lista (S1L) - Iterálás és Keresés



S1L_search(L:E1*, k:T) : E1*



S1L_search(L:E1*, k:T) : E1*



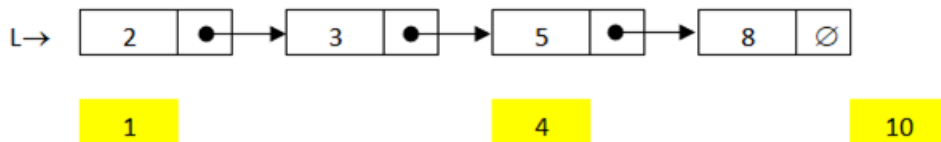
• Iterálás:

- p : mutató egy elemre, $p: E1^*$
- $p \rightarrow \text{key}$: a listaelem értéke.
- $p \rightarrow \text{next}$: a következő elem mutatója (ha van).
- Új elem allokalása: $p := \text{new } E1$
- Listaelem törlése: delete p
Törlés után bármelyik mezőre való hivatkozás érvénytelen.

• Keresés:

- Legyen **L** egy rendezett S1L lista.
Keress egy olyan elemet L-ben, amelynek a kulcsa 'k'. Ha található ilyen elem, térj vissza a mutatójával, különben térj vissza *NULL*ával.

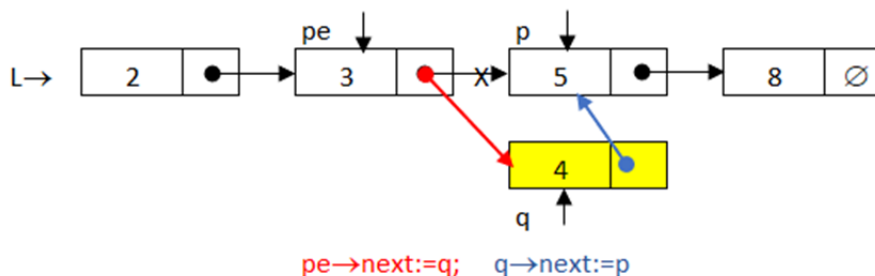
Egyszerű, egyirányú láncolt lista (S1L) - Beszúrás



- Beszúrás:

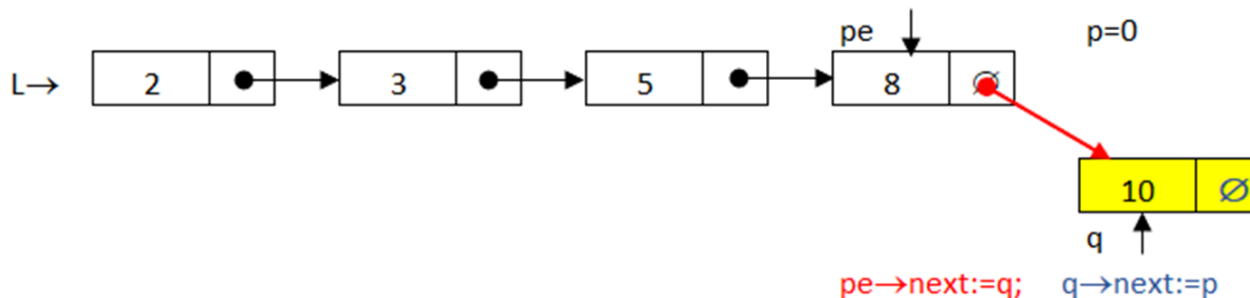
- Legyen **L** egy kulcs szerint szigorúan monoton növekvően rendezett S1L (egyirányú, fejelem nélküli lista). Szűrjünk be a listába egy **k** kulcsot a rendezettség szerinti helyére.
- Ha ilyen kulcs már létezik, ne történjen semmi. (halmaz)
- A kulcs helye **négy** lehetséges helyen lehet:
 - 1) Az elem a lista belsejébe illeszkedik,
 - 2) Az elem a legnagyobb, így az lesz az utolsó,
 - 3) Az elem a legkisebb, így az lesz az első,
 - 4) Üres a lista, most szűrjük be az első elemet.

Beszúrás - 1) Lista belseje



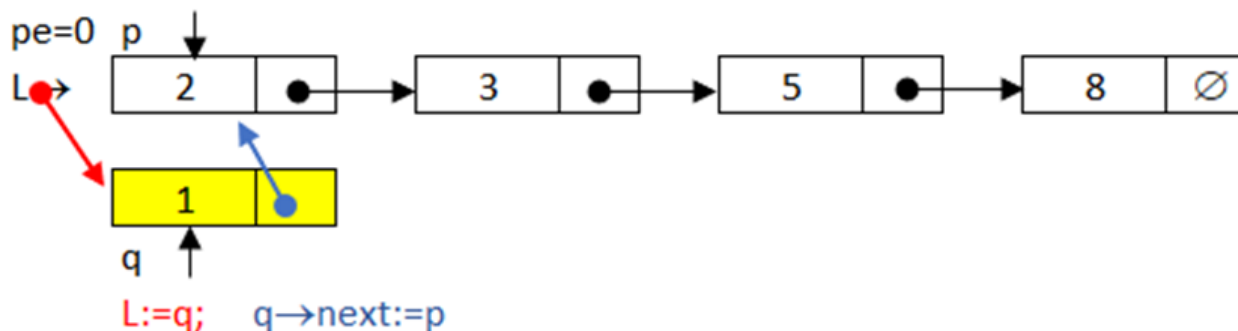
- Két bejáró pointert fogunk használni: pe , p – a kettő közé kell befűzni az új elemet.
- Lista belsejébe történő befűzés:
 - p pointerrel megkeressük az első olyan elemet, amelyik nem kisebb, mint a beszúrandó kulcs, közben pe párhuzamosan halad p -vel, mindig a p előtti elem címét tartalmazza.
 - Ha a p megáll egy listaelemen, akkor a p című elem kulcsa nagyobbbecnyelő, mint a beszúrandó kulcs.
 - Ellenőrizzük, hogy $p \rightarrow key$ nem a beszúrandó kulcs. Ha nem, akkor pe és p közé befűzzük az új elemet. (Előtte lefoglalunk az új elem számára egy új listaelemet, és kitöltjük a kulccsal, és az egyéb szükséges adatokkal.)

Beszúrás - 2) Lista vége



- A lista végén:
 - Két bejáró pointert fogunk használni: pe , p – a kettő közé kell befűzni az újelemet.
 - p pointer „leszalad” a listáról, mert a listaelemek mindegyike kisebb, mint a beszúrandó kulcs.
 - Mivel pe együtt haladt p -vel, a lista utolsó elemének címe pe -ben van, így segítségével az új listaelemet be tudjuk fűzni a lista végére.

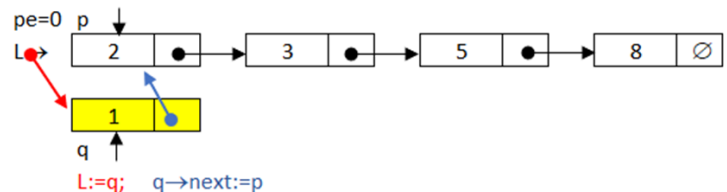
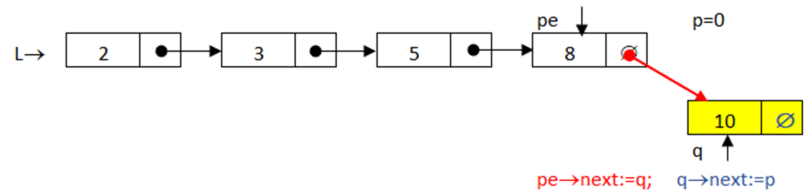
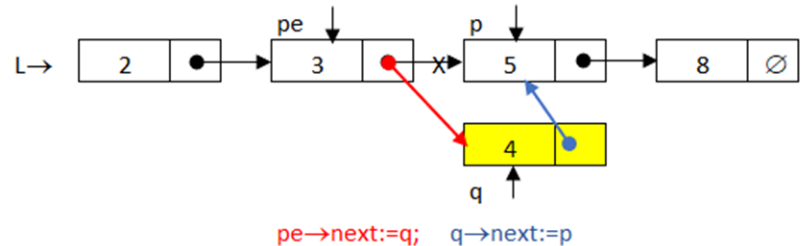
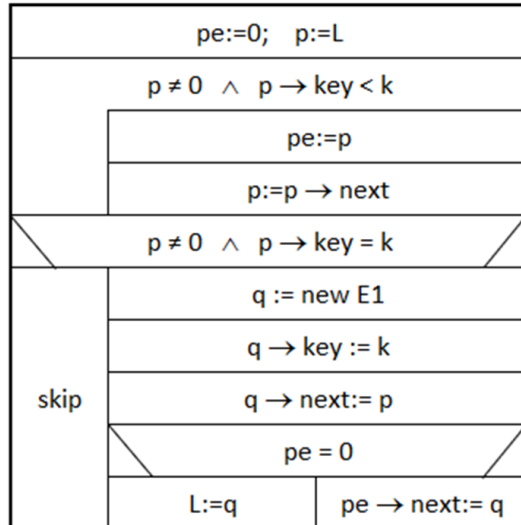
Beszúrás - 3) Lista eleje és 4) Üres lista



- A lista elején:
 - Beszúrás az elejére azt jelenti, hogy L -nek most a új elemre kell mutatnia, azaz L fog megváltozni (pe ilyenkor $NULL$)
- Üres lista esete: szintén L fog változni (pe , és p is $NULL$):

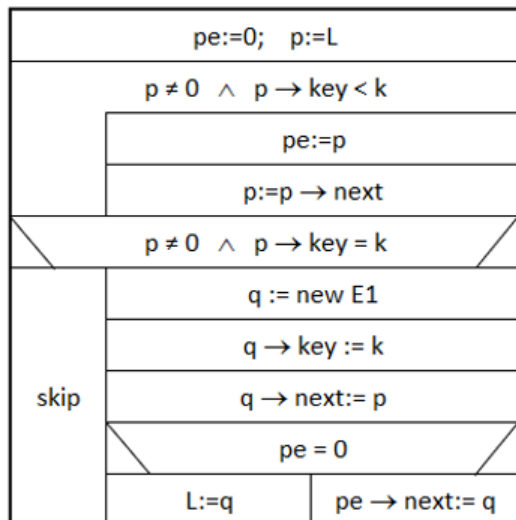
Beszúrás - Struktogram

S1L_insert(&L: E1*, k: T)



Beszúrás - Struktogram

S1L_insert(&L: E1*, k: T)



Fontos, hogy L referencia szerinti paraméter! Megváltozhat!

Bejáró pointerok indítása.

Első olyan elem megkeresése, amelyik nem kisebb.

Ha van már k kulcsú elem, készen vagyunk...

Új listaelem lefoglalása.

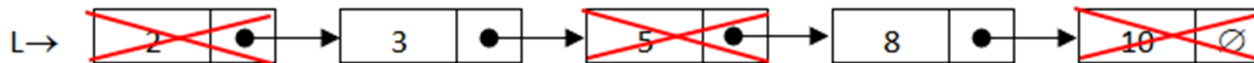
Kulcs beleírása az új elembe

A következő elem p.

Attól függően, hogy volt-e előző elem, L vagy

pe című elem next pointere változik.

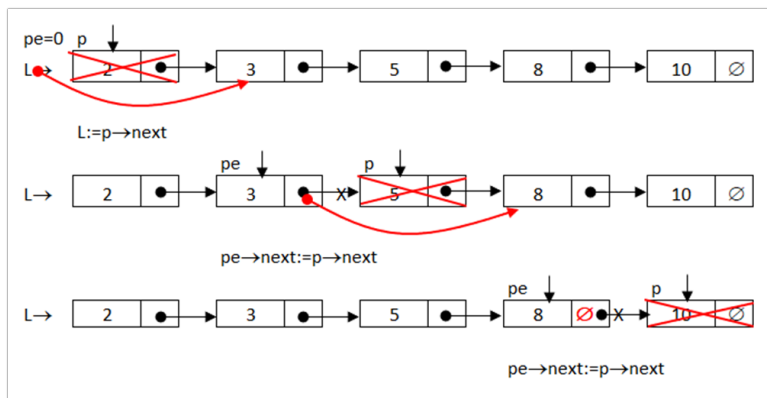
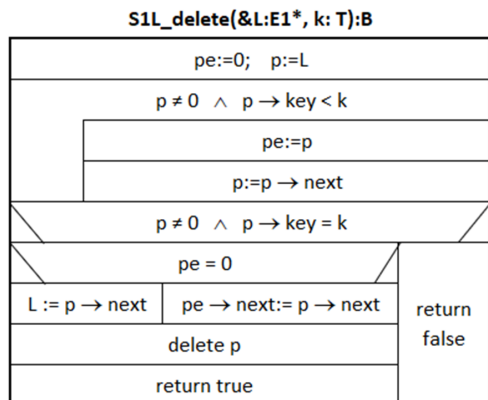
Egyszerű, egyirányú láncolt lista (S1L) - Törlés



- Törlés:

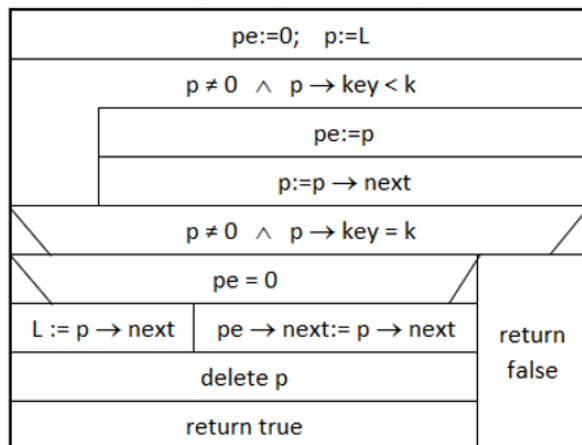
- Legyen **L** egy kulcs szerint szigorúan monoton növekvően rendezett *S1L* (egyirányú, fejelem nélküli lista). Keressük meg és töröljük a *k* kulcsú elemet, ha van.
- Ha a törlés sikeres, igazzal, ha sikertelen hamissal térjünk vissza.
- A lista egybenmaradásához szükség van az előző elem megjegyzésére is a törléshez.
- (Sikeres) törlés esetén a következő esetek lehetségesek:
 - Lista első elemét töröljük.
 - Lista belsejéből törölünk (előtte, és utána is van elem),
 - Lista legutolsó elemét töröljük (előtte van elem),

Törlés - Struktogram



Törlés - Struktogram

S1L_delete(&L:E1*, k: T):B



Fontos, hogy L referencia szerinti paraméter! Megváltozhat!

Bejáró pointerok indítása.

Első olyan elem megkeresése, amelyik nem kisebb.

Ha megtaláltuk a keresett elemet,

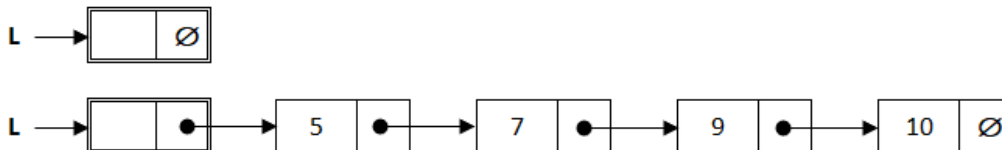
kiláncoljuk, attól függően, hogy a lista elején volt-e, vagy valahol a lista belsejében.

Kiláncolás után felszabadítjuk az elem helyét.

Fejelemes lista - Egyirányú

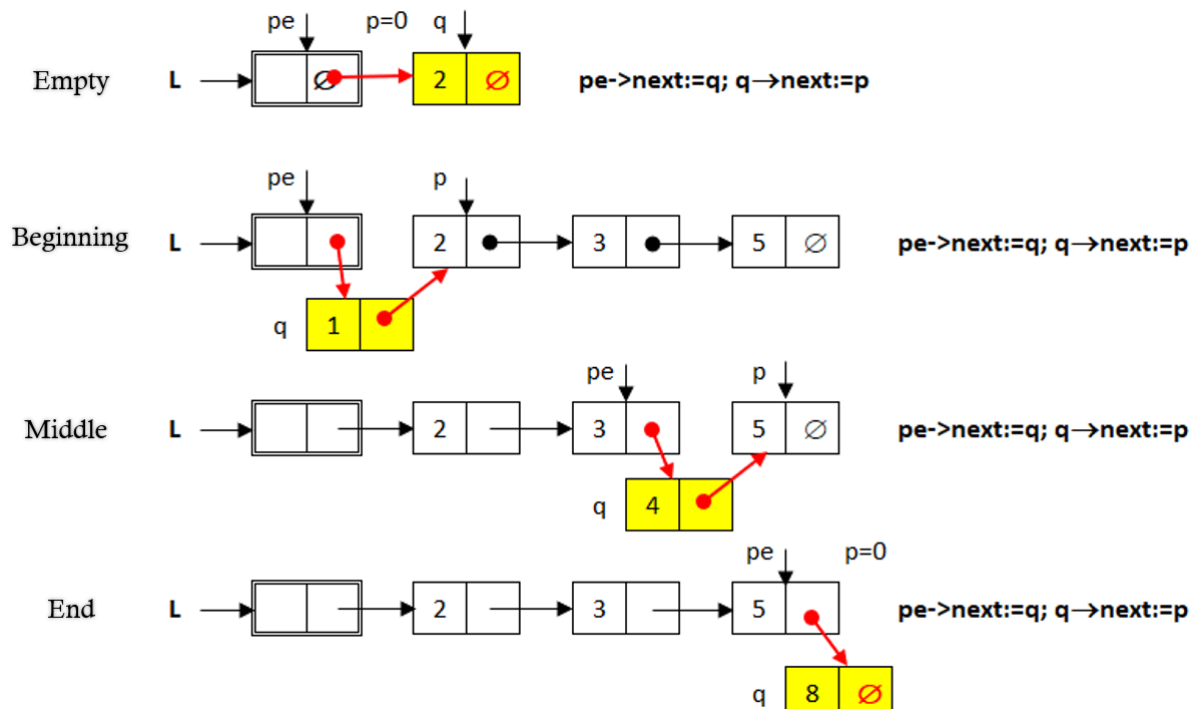
Fejelemes egyirányú láncolt lista (H1L)

- Fejelem bevezetése:
 - Az *S1L* listák kezelésénél az algoritmusnak külön kell gondoskodnia arról ha a lista elején történik a beszúrás/törlés. ($pe = NULL$)
 - Ennek a problémának a megoldására, és a felesleges elágazások elkerülésére bevezetünk egy fejelemet elemet.
 - Így a lista mindig tartalmazni fog egy E1 elemet, a fejelemet, és soha nem lesz teljesen üres!
 - A fejelemben nem tárolnak "rendes" adatot.

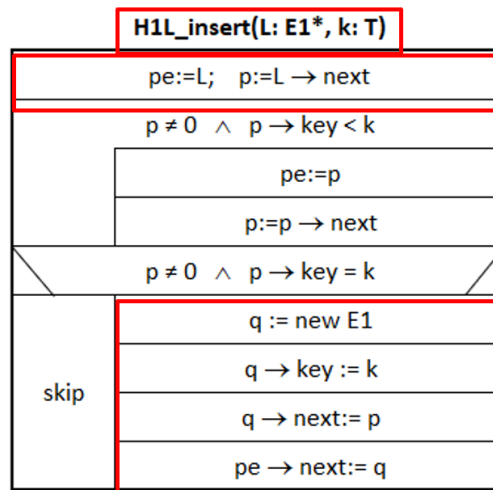
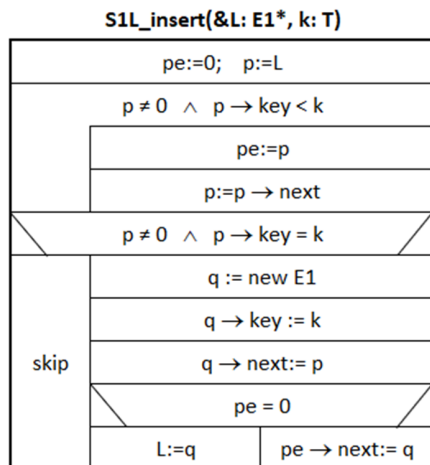


- A keresés kissé eltérő lesz:
 - Az első elemről kezdjük, ami a fejelem után következik.

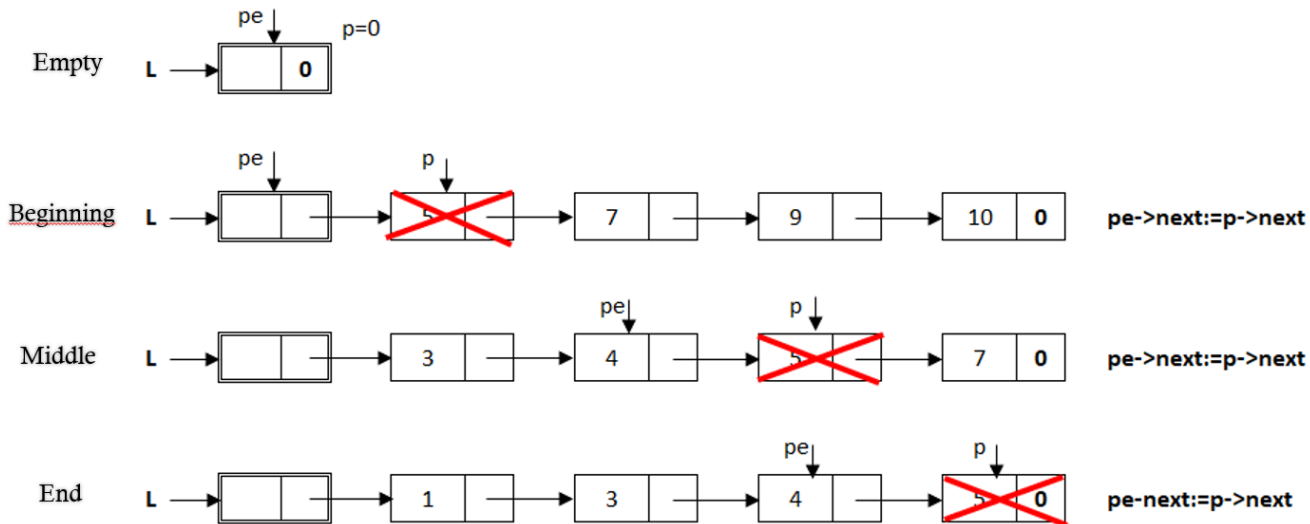
Fejelemes egyirányú láncolt lista (H1L) - Beszúrás



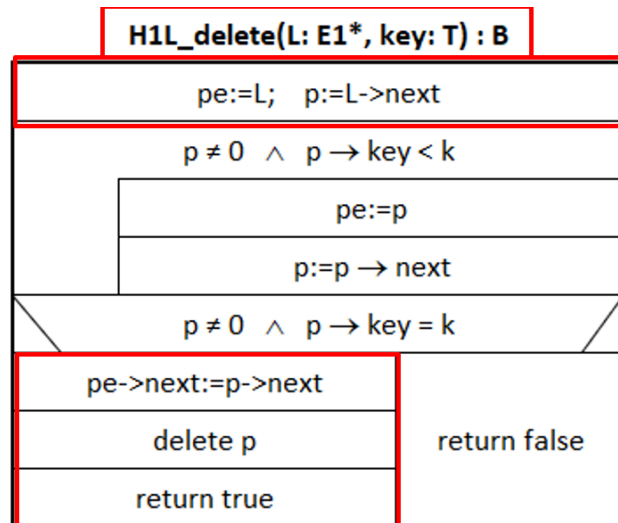
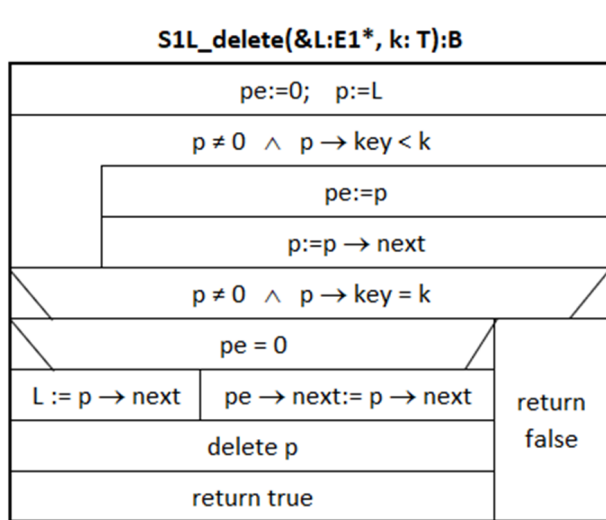
Beszúrás - Structogram



- Változások:
 - **L** nincs referencia szerint átadva.
 - Kezdés: *pe* a fejlemez, *p* az első elem (lehet *NULL*).
 - Nincs szükség ágakra, mivel *pe* soha nem *NULL*.



Törlés - Structogram



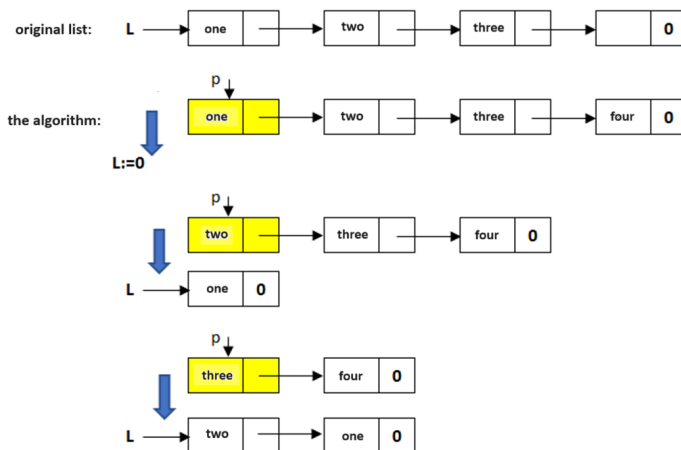
- Változások:
 - **L** nincs referencia szerint átadva.
 - Kezdés: *pe* a fejlemez, *p* az első elem (lehet *NULL*).
 - Nincs szükség ágakra, mivel *pe* soha nem *NULL*.

Feladatok

S1L lista megfordítása

- Legyen **L** egy S1L. Fordítsuk meg az elemek sorrendjét.
- Bemenet:
 - **L** $\rightarrow$ egy $\rightarrow$ két $\rightarrow$ három $\rightarrow$ négy
- Kimenet:
 - **L** $\rightarrow$ négy $\rightarrow$ három $\rightarrow$ két $\rightarrow$ egy
- Ötlet:
 - "Dekonstruáljuk" az eredeti listát, és p mutat az első elemre.
 - Állítsuk L-t üres listává //L:=0
 - Iteráljunk az eredeti listán, és minden egyes elemet szúrjunk be **L** elejére, hogy megkapjuk a fordított listát.

S1L lista megfordítása



ReverseList(&L:E1*)

p := L
L := 0
p ≠ 0
q := p
p := p->next
q->next := L
L := q

- q az az elem, amelyet az új elejére kell beszúrni
- Lépünk a következő elemre az előző listában, különben elveszítenénk a kapcsolatot az eredeti listával
- q beszúrása: az előző első elemnek q után kell következnie, és q-nak kell lennie az új első elemnek.

S1L lista megfordítása

Megfordít(&L:E1*)

$p := L$
$L := 0$
$p \neq 0$
$q := p$
$p := p \rightarrow \text{next}$
$q \rightarrow \text{next} := L$
$L := q$

Fontos, hogy L címszerinti paraméter

Indulás: p az eredeti lista első elemére mutat

L listát üresre állítjuk

Amíg az eredeti lista el nem fogy:

q lesz az az elem, amelyet most átfűzünk L elejére,

mivel q beláncolásakor elveszíténénk a q utáni elemet, p-vel tovább kell lépni!

q beláncolásának lépései: q után L eddigi első eleme következzen,

L lista új első eleme legyen a q.

H1L szétfűzése két listába

- Adott egy $H1L$, egész számokat tartalmaz, a fejelemre **L1** mutat.
- Fűzzük ketté az elemeket: **L1**-ben maradjanak a páros elemek, egy új **L2** $H1L$ listába fűzzük át a páratlan elemeket.
- Az eredeti sorrendet tartsuk meg, azaz átfűzésnél mindig a lista végére kell majd fűzni.
- Az **L1** listát egyszer lehet bejárni, **L2**-be a befűzés konstans műveletigényű legyen!

H1L szétfűzése két listába

Szétfűz(L1:E1*): E1*

L2 := new E1 //L2 → next=0	
u:=L2	
pe:= L1; p:=L1 → next	
p ≠ 0	
p → key mod 2 = 0	
pe:=p	pe → next:= p → next
p:= p → next	u → next:= p
	p → next:= 0
	u:=p
	p:= pe → next
return L2	

Első lépésként egy új H1L létrehozását mutatjuk be. Kihasználjuk, hogy E1 konstruktora az elem next pointerét 0-ra állítja!
u mindig L2 utolsó elemére fog mutatni, ez kezdetben a fejelem.
pointerek L1 bejárásához

Ha páratlan kulcsú elemmel találkozunk, a lépések:

p kifűzése

p befűzése L2 végére

mivel p az L2 utolsó eleme, 0-ra állítjuk a next pointerét

módosítjuk u-t

p-vel L1 következő elemére állunk

Primszita (Sieve of Eratosthenes) egyszerű listán

- Az Eratoszthenészi szita algoritmus mintájára készítsünk olyan S1L listát, mely 2-től n -ig a prímszámokat tartalmazza.
- Ötlet:
 - Eratoszthenész Pentatlosz talált egy nagyon hatékony módszert egy nem túl nagy sokaságból a prímszámok kiválogatására.
 - Írjuk fel a listára a természetes számokat 1-től, addig ameddig a prímeket keressük. 1 nem prím, húzzuk ki. 2 az első prím. 2-től indulva minden második elemet húzzunk ki (mivel ezek 2 többszörösei, tehát nem prímszámok).
 - A fennmaradó elemek közül a következő a 3, ez egy prímszám, tartsuk meg, és töröljük a 3 többszöröseit.
 - És így tovább, amíg el nem érjük az utolsó elemet.
 - A szitában fennmaradt számok prímek.

Primszita - Példa

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

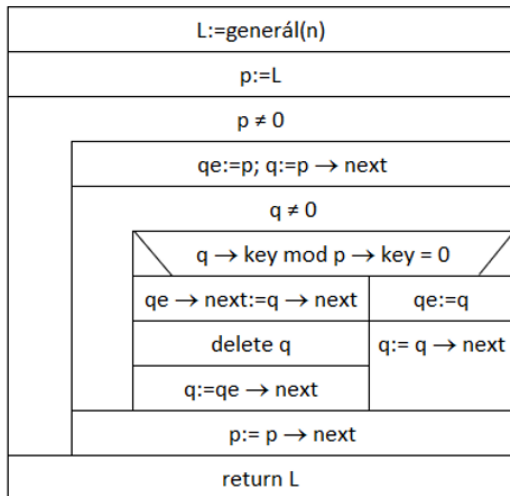
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
--------------	---	---	--------------	---	--------------	---	--------------	--------------	---------------	----	---------------	----	---------------	---------------	---------------	----	---------------	----	---------------

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
--------------	---	---	--------------	---	--------------	---	--------------	--------------	---------------	----	---------------	----	---------------	---------------	---------------	----	---------------	----	---------------

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
--------------	---	---	--------------	---	--------------	---	--------------	--------------	---------------	----	---------------	----	---------------	---------------	---------------	----	---------------	----	---------------

Primszita - Struktogram 1/2

Primszita(n:N) : E1*



Generálunk egy egyszerű listát 2..n természetes számokból.

p mindig a következő, még a szitában lévő prímszámmra mutat

(elsőként a 2-re). Amíg ez nem 0 ...

q-val a p utáni elemeket fogjuk bejárni, és kifűzzük p → key többszöröseit

Ha p → key osztója q → key-nek, a q című elemet

töröljük a láncból. Ehhez szükségünk

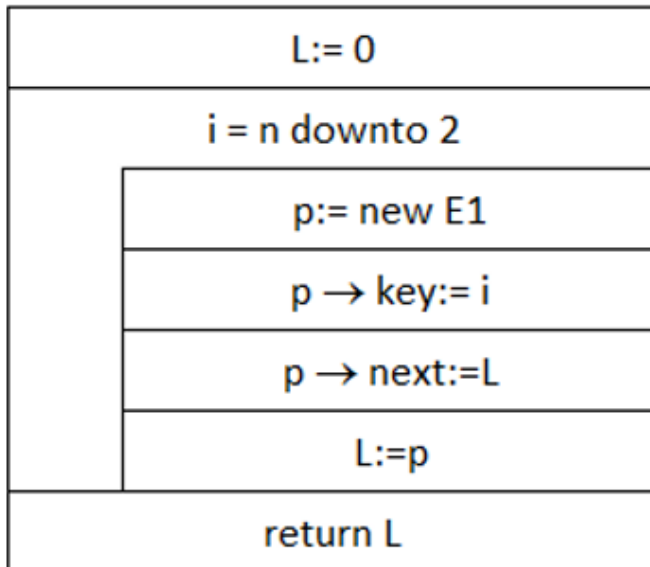
van q elem előzőjére, ez lesz a qe pointerben.

ha p című elem többszöröseit töröltük, a következő

elem megint prím lesz.

Primszita - Struktogram 2/2

generál(n:N) : E1*



*Egyszerű lista feltöltése 2..n természetes számokkal.
Trükk: a ciklust csökkenőleg futtatjuk, így fordított sorrendben generáljuk a számokat, tehát mindig a lista elejére kell befűzni az új elemet.*

Köszönöm a figyelmet!